method_missing is there, because it's a private instance method of BasicObject that every object inherits.

You can experiment by calling method_missing yourself. It's a private method, but you can get to it through send:

```
nick.send :method_missing, :my_method
```

```
NoMethodError: undefined method `my_method' for #<Lawyer:0x007f801b0f4978>
```

You have just done exactly what Ruby does. You told the object, "I tried to call a method named my_method on you, and you did not understand." BasicObject#method_missing responded by raising a NoMethodError. In fact, this is what method_missing does for a living. It's like an object's dead-letter office, the place where unknown messages eventually end up (and the place where NoMethodErrors come from).

Overriding method_missing

Most likely, you will never need to call method_missing yourself. Instead, you can override it to intercept unknown messages. Each message landing on method_missing's desk includes the name of the method that was called, plus any arguments and blocks associated with the call.

methods/more_method_missing.rb

```
class Lawyer
  def method_missing(method, *args)
    puts "You called: #{method}(#{args.join(', ')})"
    puts "(You also passed it a block)" if block_given?
  end
end

bob = Lawyer.new
bob.talk_simple('a', 'b') do
  # a block
end
```

```
You called: talk_simple(a, b)
(You also passed it a block)
```

Overriding method_missing allows you to call methods that don't really exist. Let's take a closer look at these weird creatures.

Ghost Methods

When you need to define many similar methods, you can spare yourself the definitions and just respond to calls through method_missing. This is like saying to the object, "If they ask you something and you don't understand, do this."